Perguntas em aberto

Registro exigido por C-12. Mudança aprovada entra primeiro na Spec, depois nas features, depois no código.

Estado em 11/11/2026: nenhuma pergunta aberta. Q-01 a Q-06 foram decididas (Spec v1.1); Q-07 a Q-10 surgiram na T-19, Q-11 a Q-14 na T-24 e Q-15 a Q-19 no mapeamento de bugs de 11/11; todas decididas (Spec v1.2).

Decididas

ID	Pergunta	Decisão	Por quê	Efeito
Q-01	Ao conceder de novo após revogar, `DataRevogacao` é apagada ou mantida?	**Mantida** como registro da última revogação	Rastro de auditoria: a LGPD exige demonstrar quando o titular revogou; apagar perderia a prova. O status já diz se vale	Nenhuma mudança de código
Q-02	Se nenhum segmento ativo casar (sem o `padrao`), qual erro?	**Falha de configuração** (`ErrSemSegmentoAplicavel`) → HTTP 500, não 422	O cliente não enviou nada inválido; quem errou foi a configuração dos segmentos. 422 culparia o cliente	`SegmentacaoService` alterado; testes unitários novos. O `padrao` é garantido por migração (Generic) e pelos dados de demonstração
Q-03	O que faz o `IntegracaoGateway.BuscarEventos(desde)`?	**Fora do MVP**: continua stub vazio; eventos entram só por `POST /v1/clientes/{id}/eventos` (D-12)	Não há sistema de e-commerce/CRM real para integrar; inventar o contrato agora seria especulação. A rota de eventos já cobre a coleta	Nenhuma mudança de código; requisito futuro
Q-04	Atores de UC-08 e "Agente Inteligente" no UML1	**Confirmados**: UC-08 com Cliente (titular) e Administrador; Agente Inteligente como ator de automação de UC-03/UC-04; UC-09 com Cliente	Mantém o que o diagrama original já tinha (Administrador) e torna explícita a automação	Já no Mermaid do UML1 e na imagem `docs/uml/img/v1/UML1_casos_de_uso.png`
Q-05	A regra de recomendação (primeiro produto ativo da categoria, por nome) vai para a apresentação final?	**Sim, mantida no MVP**	"Mais vendido" exige saber o produto de cada compra, e o evento não tem `produtoId` no modelo; a regra atual é determinística e testável. Evolução fica pós-MVP	Nenhuma mudança de código
Q-06	`PUT /consentimento` com consentimento já CONCEDIDO: 409 ou 200 idempotente?	**`409 transicao_invalida`**, como na máquina de estados	Mantém um único comportamento entre domínio e API (UML5); o cliente consulta `GET /consentimento` antes. Trocar a versão do termo exige revogar e conceder de novo, o que deixa o histórico explícito	Nenhuma mudança de código
Q-07	A Spec 3.8 e o UML8 dizem que `DATABASE_URL` é obrigatória a partir da fase Generic, mas a D-23, o `make serve` e o BDD dependem do modo memória. Qual vale?	**Vazia continua = modo memória** (desenvolvimento, BDD e demo local); **preenchida = PostgreSQL**. A implantação do UML8 (compose) sempre a define	Mantém o BDD e a demo rápidos e sem banco (C-10: testes unitários sem rede nem banco) e deixa a implantação fiel ao UML8	Spec 3.8, D-23 e UML8 ajustados; `omega-server` passa a montar os `Pg*Repository` quando há URL
Q-08	No PostgreSQL, de onde vêm os segmentos `premium` / `engajado` e o catálogo de produtos? Não há rota para cadastrá-los (gestão pela API está fora do MVP)	**Migração própria de dados sintéticos** (`002_dados_demonstracao.sql`), com os mesmos IDs do modo memória. O `padrao` fica na migração inicial, como manda a Spec	"Segmentos entram por migração" (Spec 1); o catálogo segue o mesmo caminho por não ter outro. IDs iguais deixam memória e banco com o mesmo comportamento observável	Duas migrações; `memoria.CarregarDemonstracao` e a 002 mantêm os mesmos dados (um teste compara). Aceito no MVP: todo banco migrado recebe os dados sintéticos; fora do MVP, a 002 vira opcional
Q-09	Com duas réplicas (UML8), quem aplica as migrações?	**O próprio `omega-server` ao subir**, sob `pg_advisory_lock`, registrando cada versão em `schema_migracoes`	Evita um terceiro artefato ou subcomando fora do UML8; a trava impede que as duas réplicas migrem ao mesmo tempo e a tabela torna a operação idempotente	Nenhum subcomando novo

ID	Pergunta	Decisão	Por quê	Efeito
Q-10	Qual driver para "`database/sql` com driver compatível"?	**`github.com/jackc/pgx/v5/stdlib`**	Driver PostgreSQL mais mantido em Go, usado pela interface padrão `database/sql` exigida pela Spec; só `infra/repository/pg` o importa (teste de arquitetura passa a cobrir `infra`)	Nova dependência só na infraestrutura. Versão fixada em **v5.6.0** (com `golang.org/x/crypto v0.17.0`), a mais recente compatível com o Go 1.22 do CI; subir exige Go ≥ 1.23 no CI
Q-11	O cenário "as duas réplicas atenderam" precisa saber qual réplica respondeu, mas a API não expõe isso	**O Nginx devolve o cabeçalho `X-Omega-Replica` com o endereço da réplica** (`$upstream_addr`)	Fica na infraestrutura (UML8): nenhuma rota, campo ou DTO novo na API (C-04)	Só `nginx.conf`
Q-12	TLS no Nginx (Spec 3.8) sem certificado da equipe	**Certificado autoassinado gerado na primeira subida**, num volume; um certificado real pode ser montado no mesmo caminho. Só a porta HTTPS é publicada (`PERFILA_PORTA_HTTPS`, padrão 8443)	Demonstração reproduzível em qualquer máquina, sem segredo no repositório; "única porta de entrada" do UML8	`deploy/nginx/`
Q-13	`@implantacao` sobe contêineres: entra no `make verify`?	**Não.** O filtro padrão passa a ser `~@slow && ~@wip`; `@implantacao` roda em `make bdd-implantacao` e num job próprio do CI	Mantém o portão de toda tarefa rápido e sem Docker; o plano já previa job separado	`features/bdd_test.go`, `Makefile`, CI
Q-14	Senha do banco e `AUTH_TOKEN` da implantação	**Gerados aleatoriamente** por `scripts/preparar-implantacao.sh` em `deploy/.env` e `deploy/config.env` (ignorados pelo git); a `DATABASE_URL` é montada pelo compose. Logs do Nginx sem IP do cliente e sem query string	Nenhum segredo versionado; IP é dado pessoal na LGPD, então os logs seguem C-09 também no balanceador	`deploy/`, `scripts/`
Q-15	Compras com valor perto do limite do `float64` (ex.: `1e308`) estouram a soma: `TicketMedio` vira infinito, o banco grava `Infinity` e o processamento responde 200 com corpo vazio	**`Valor` no máximo 1.000.000** (RN-04); a API serializa a resposta antes de escrever o status e devolve 500 se falhar	O limite impede o estouro na origem; a serialização prévia garante que nenhuma falha vire "200 vazio"	RN-04, `evento.feature`, `Validar`, `escreverJSON`
Q-16	Evento com `Timestamp` segundos à frente do servidor (relógio do dispositivo adiantado) é rejeitado	**Tolerância de 2 minutos** em RN-04. O score (RN-05) continua contando só eventos até agora	Relógios de clientes divergem; rejeitar perde dados legítimos. 2 minutos cobrem a deriva comum sem aceitar datas absurdas	RN-04, `evento.feature`, `Validar`
Q-17	Critério inválido num segmento da configuração, ou segmento do perfil inexistente, responde 422 `dados_invalidos`, culpando o cliente	**Falha de configuração (500)**, `ErrSegmentoMalConfigurado`, como a Q-02	Mesmo raciocínio da Q-02: o cliente não enviou nada inválido	RN-06, `segmentacao.feature`, `SegmentacaoService`, UC-04
Q-18	Cada processamento cria mais uma recomendação ATIVA, e as anteriores nunca expiram enquanto válidas	**No máximo uma ATIVA por cliente**: UC-04 expira as anteriores antes de gravar a nova (novo método `ListarAtivasPorCliente`)	Evita recomendações contraditórias ativas ao mesmo tempo e mantém o UML6 (ATIVA → EXPIRADA por `Expirar`)	RN-08, port, `recomendacao.feature`, memória, PostgreSQL e contrato
Q-19	`GET /saude` responde 200 com o banco fora do ar; corpo com JSON extra depois do objeto é aceito	**`/saude` consulta o banco** (503 `indisponivel` se não responder); **corpo deve ser um único objeto JSON**	O balanceador e o healthcheck passam a enxergar a falha real; o corpo ambíguo vira `dados_invalidos`	Spec 3.7, `api.feature`, `api`

Decisões de modelagem (T-04)

As decisões D-01 a D-23 da Spec (seção 3.9) foram **adotadas como finais em 07/11/2026**. Qualquer mudança futura segue o fluxo Spec → features → código.